

Trasformazione di Vista (Viewing 3D)





Cambio di Sistema di Riferimento

Problema: sia dato un Sistema di Riferimento (frame) cartesiano $(\underline{e1}, \underline{e2}, \underline{e3}, O)$ e un punto p in questo sistema. Sia poi dato un nuovo frame $(\underline{u}, \underline{v}, \underline{w}, O')$. Si determinino le coordinate di p rispetto al nuovo frame.

Vediamo un modo per risolvere il Problema.

Dobbiamo determinare le coordinate $p_{new} = [u, v, w, 1]$ così che

$$[\underline{u}, \underline{v}, \underline{w}, O'] \begin{pmatrix} u \\ v \\ w \\ 1 \end{pmatrix} = [\underline{e1}, \underline{e2}, \underline{e3}, O] \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad (1)$$



Cambio di Sistema di Riferimento

Sappiamo che $\underline{u}$, $\underline{v}$, $\underline{w}$ sono vettori linearmente indipendenti e formano una base per R^3 . Scriviamo i vettori $\underline{e1}$, $\underline{e2}$ ed $\underline{e3}$ in questa base:

$$\underline{e1} = [1, 0, 0, 0]^T = a_{11}\underline{u} + a_{21}\underline{v} + a_{31}\underline{w}$$

$$\underline{e2} = [0, 1, 0, 0]^T = a_{12}\underline{u} + a_{22}\underline{v} + a_{32}\underline{w}$$

$$\underline{e3} = [0, 0, 1, 0]^T = a_{13}\underline{u} + a_{23}\underline{v} + a_{33}\underline{w}$$



Cambio di Sistema di Riferimento

Poiché $O-O'$ è un vettore, possiamo trovare la sua rappresentazione nella base $\underline{u}$, $\underline{v}$, $\underline{w}$

$$O-O' = [0,0,0,1] - O' = a_{14}\underline{u} + a_{24}\underline{v} + a_{34}\underline{w}$$

e quindi:

$$O = [0,0,0,1]^T = a_{14}\underline{u} + a_{24}\underline{v} + a_{34}\underline{w} + O'$$

mettendo quest'ultima insieme alle precedenti si ha:

$$[\underline{e1}, \underline{e2}, \underline{e3}, O] = [\underline{u}, \underline{v}, \underline{w}, O'] \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (2)$$

sostituendo la (2) nella relazione (1) si ha:

Cambio di Sistema di Riferimento

$$[\underline{u}, \underline{v}, \underline{w}, O'] \begin{pmatrix} u \\ v \\ w \\ 1 \end{pmatrix} = [\underline{e1}, \underline{e2}, \underline{e3}, O] \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = [\underline{u}, \underline{v}, \underline{w}, O'] \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

da cui risulta:

$$\begin{pmatrix} u \\ v \\ w \\ 1 \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

A

Quanto ottenuto dice che il cambio di sistema di riferimento può essere rappresentato da una matrice 4x4



Cambio di Sistema di Riferimento

Ovviamente si devono determinare i coefficienti a_{ij} per poter dire di aver risolto il Problema.

Le prime tre colonne sono le coordinate dei vettori $\underline{e1}$, $\underline{e2}$ ed $\underline{e3}$, nel nuovo sistema, mentre la quarta sono le coordinate di O nel nuovo sistema.

Solitamente, nella pratica, sono note le coordinate di $\underline{u}$, $\underline{v}$, $\underline{w}$ e O' rispetto al primo sistema, cioè:

$$[\underline{u}, \underline{v}, \underline{w}, O'] = [\underline{e1}, \underline{e2}, \underline{e3}, O] \begin{pmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ 0 & 0 & 0 & 1 \end{pmatrix} \leftarrow B$$

Allora la cercata matrice A sarà data da B^{-1} .

Definizione oggetto mesh 3D

In un sistema di riferimento Cartesiano $xyzO$ destrorso, consideriamo un oggetto mesh 3D dando la lista dei suoi Vertici (coord. floating point) e la lista delle sue Facce (piane).

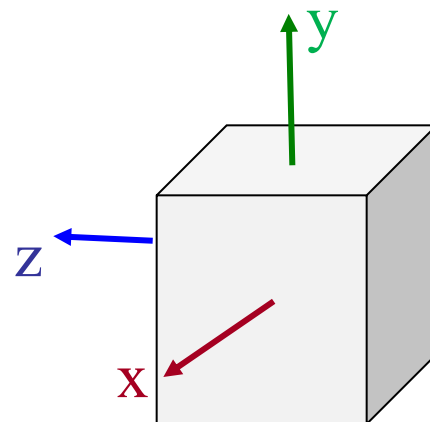
Vogliamo analizzare cosa si deve fare per ottenere una sua rappresentazione grafica 2D (immagine 2D).

Coordinate (X,Y,Z) dei Vertices

```
1> 1.000000 -1.000000 1.000000
2> -1.000000 -1.000000 1.000000
3> -1.000000 1.000000 1.000000
4> 1.000000 1.000000 1.000000
5> 1.000000 -1.000000 -1.000000
6> -1.000000 -1.000000 -1.000000
7> -1.000000 1.000000 -1.000000
8> 1.000000 1.000000 -1.000000
```

Indici dei Vertices di ogni Face

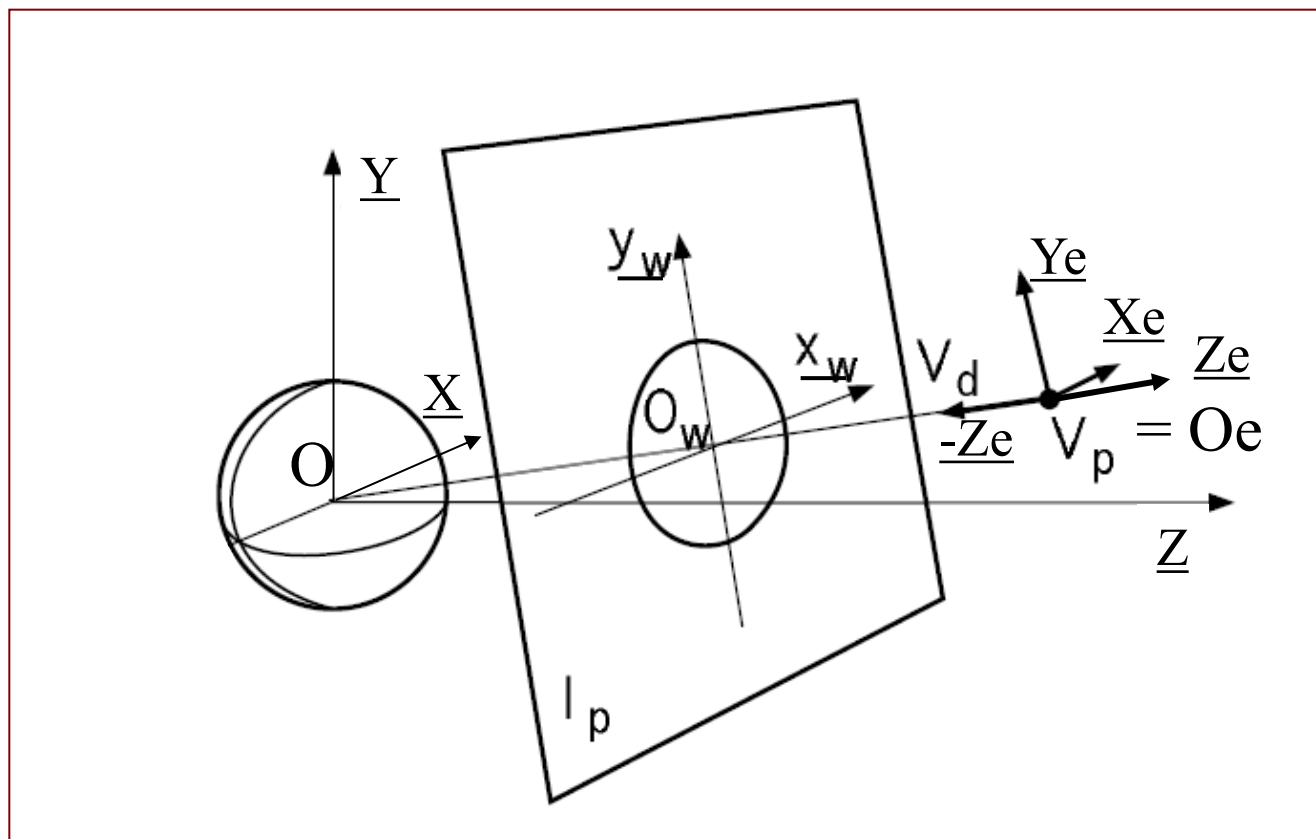
```
1> 4 3 2
2> 4 2 1
3> 8 7 3
4> 8 3 4
5> 7 8 5
...
```



Geometria e Topologia

- **definizione geometrica** (dove sono posizionati nello spazio 3D i vertici)
- **definizione topologica** (come sono connessi i vertici da lati e facce)

Sistemi di Riferimento



$\underline{X}$ $\underline{Y}$ $\underline{Z}$ $\underline{O}$: Sistema di Riferimento del Mondo (destorso)

$\underline{x}_e$ $\underline{y}_e$ $\underline{z}_e$ $\underline{O}_e$: Sistema di Riferimento (destorso) dell'
Osservatore ($\underline{O}_e$ = V_p = View Point)

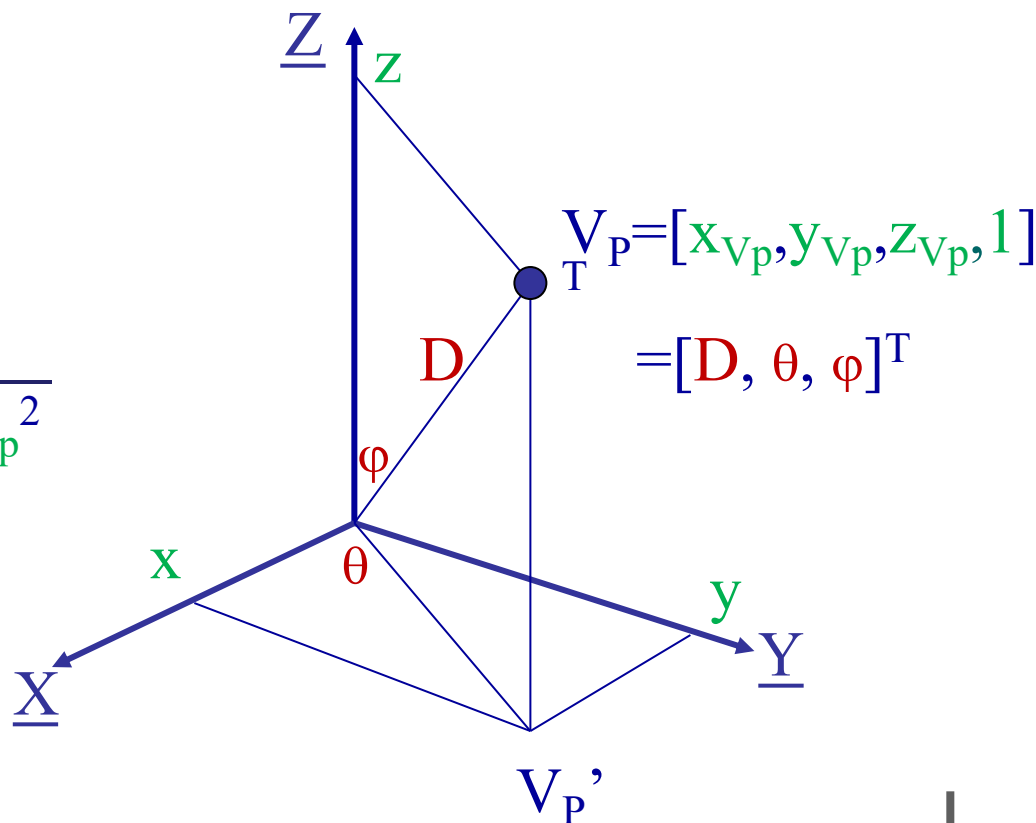
$\underline{x}_w$ $\underline{y}_w$ $\underline{O}_w$: Sistema di Riferimento 2D (Window)

Coordinate Cartesiane e Sferiche

In certe situazioni è comodo dare il View Point in coordinate sferiche ed in questo caso è bene ricordare come passare da coordinate sferiche a cartesiane e viceversa.

$$\begin{cases} x_{Vp} = D \sin(\varphi) \cos(\theta) \\ y_{Vp} = D \sin(\varphi) \sin(\theta) \\ z_{Vp} = D \cos(\varphi) \end{cases}$$

$$\begin{cases} D = \sqrt{x_{Vp}^2 + y_{Vp}^2 + z_{Vp}^2} \\ \theta = \arctan(y_{Vp} / x_{Vp}) \\ \varphi = \arccos(z_{Vp} / D) \end{cases}$$





Trasformazione di Vista

Scomponiamo la Trasformazione di Vista in due passi:

1. Cambio o Trasformazione del Sistema di Riferimento

da $\underline{X} \ \underline{Y} \ \underline{Z} \ O$ a $\underline{X_e} \ \underline{Y_e} \ \underline{Z_e} \ O_e$

2. Proiezione Geometrica

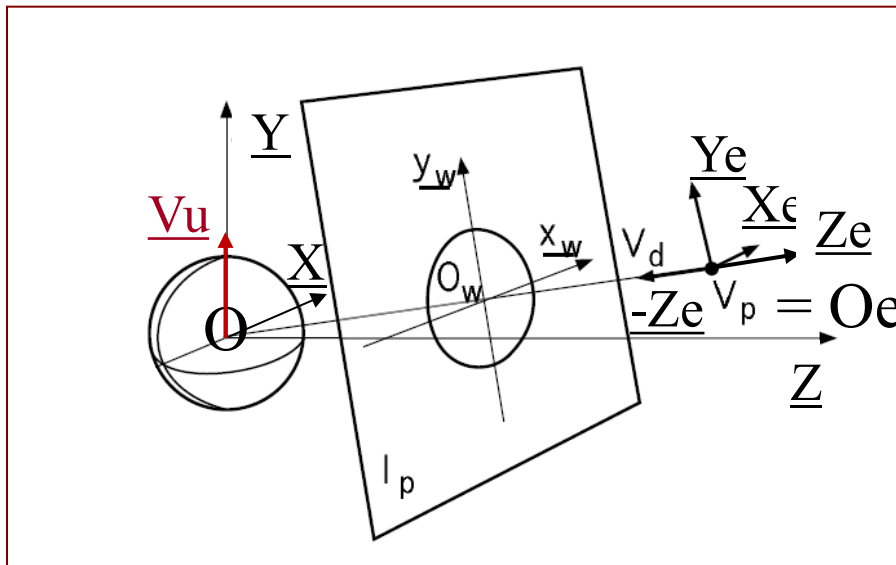
da $\underline{X_e} \ \underline{Y_e} \ \underline{Z_e} \ O_e$ a $\underline{X_w} \ \underline{Y_w} \ \underline{O_w}$

Trasformazione Sistema di Riferimento e View-Up vector (Vu)

$$\underline{V_p} = [D, \theta, \varphi, 1]^T = [D \sin\varphi \cos\theta, D \sin\varphi \sin\theta, D \cos\varphi, 1]^T$$

$$[x_e, y_e, z_e, 1]^T = \underline{V} \cdot [x, y, z, 1]^T$$

V è detta
ViewMatrix



$$\underline{Z_e} = \underline{V_d} / \|\underline{V_d}\| = -(\underline{V_p} - \underline{O}) / \|\underline{V_p} - \underline{O}\|$$

$$\underline{X_e} = (\underline{Z_e} \times \underline{V_u}) / \|\underline{Z_e} \times \underline{V_u}\|$$

$$\underline{Y_e} = -(\underline{Z_e} \times \underline{X_e}) / \|\underline{Z_e} \times \underline{X_e}\|$$

e poi: $\underline{Z_e} = -\underline{Z_e}$

Vu viene definito nel Sistema di Riferimento del Mondo



Trasformazione Sistema di Riferimento e View-Up vector (Vu)

$$V_p = [D, \theta, \varphi, 1]^T = [D \sin\varphi \cos\theta, D \sin\varphi \sin\theta, D \cos\varphi, 1]^T$$

$$B = \begin{pmatrix} \boxed{\underline{X_e}} & \boxed{\underline{Y_e}} & \boxed{\underline{Z_e}} & D \sin\varphi \cos\theta \\ 0 & 0 & 0 & D \sin\varphi \sin\theta \\ 0 & 0 & 0 & D \cos\varphi \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$V = B^{-1}$$

$$[x_e, y_e, z_e, 1]^T = V \cdot [x, y, z, 1]^T$$



Libreria m4.js

Per definire la matrice di vista V useremo una function, della libreria **m4.js**, chiamata **lookAt** con i seguenti parametri:

```
function lookAt( cameraPosition, target, vu ) {...}
```

cameraPosition: posizione della camera

target: punto in cui la camera guarda

vu: view-up vector

poi si calcola la sua inversa:

```
function inverse( matrix ) {...}
```

Esempio:

```
// Compute the camera's matrix using look at.  
var cameraMatrix = m4.lookAt(cameraPos, cameraTar, vu);  
// Make a view matrix from the camera matrix.  
var viewMatrix = m4.inverse(cameraMatrix);
```



Funzioni della Libreria m4.js

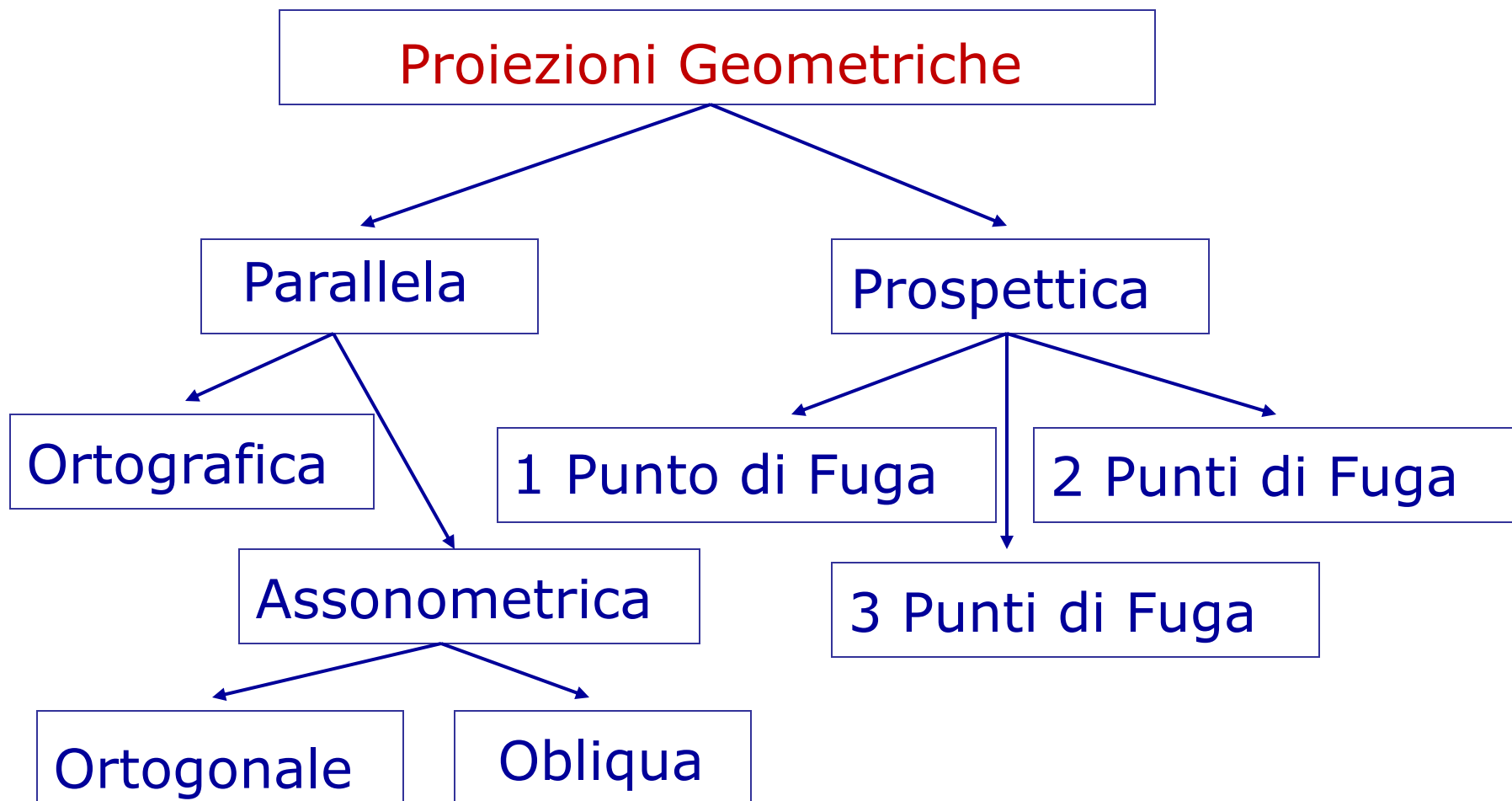
copy,
lookAt,
addVectors,
subtractVectors,
distance,
distanceSq,
normalize,
compose,
cross,
decompose,
mvec4,
dot,
identity,
transpose,
length,
orthographic,
frustum,

perspective,
translation,
translate,
xRotation,
yRotation,
zRotation,
xRotate,
yRotate,
zRotate,
axisRotation,
axisRotate,
scaling,
scale,
flatten,
multiply,
inverse,
transformVector,

transformPoint,
transformDirection,
transformNormal,
setDefaultType

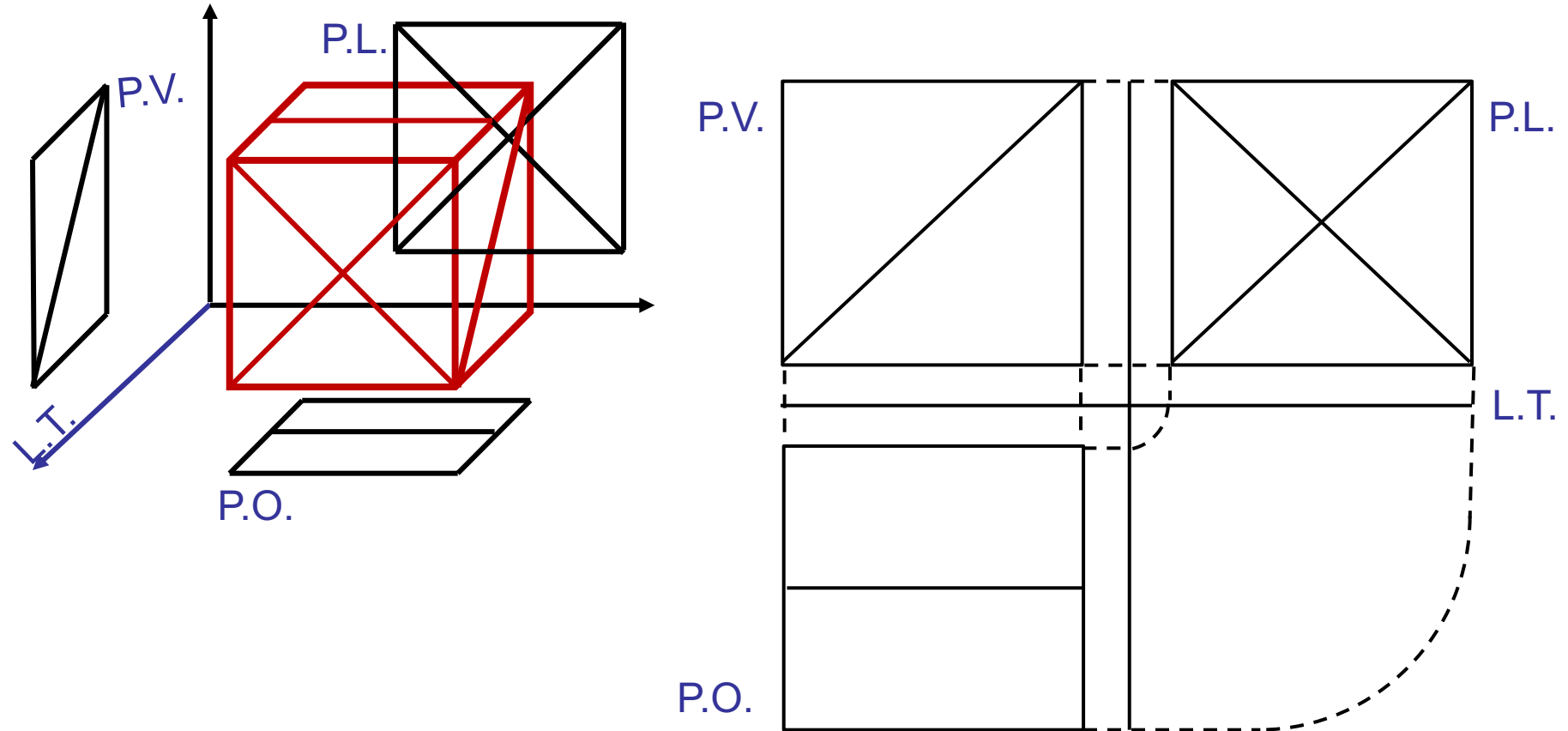


Proiezioni Geometriche





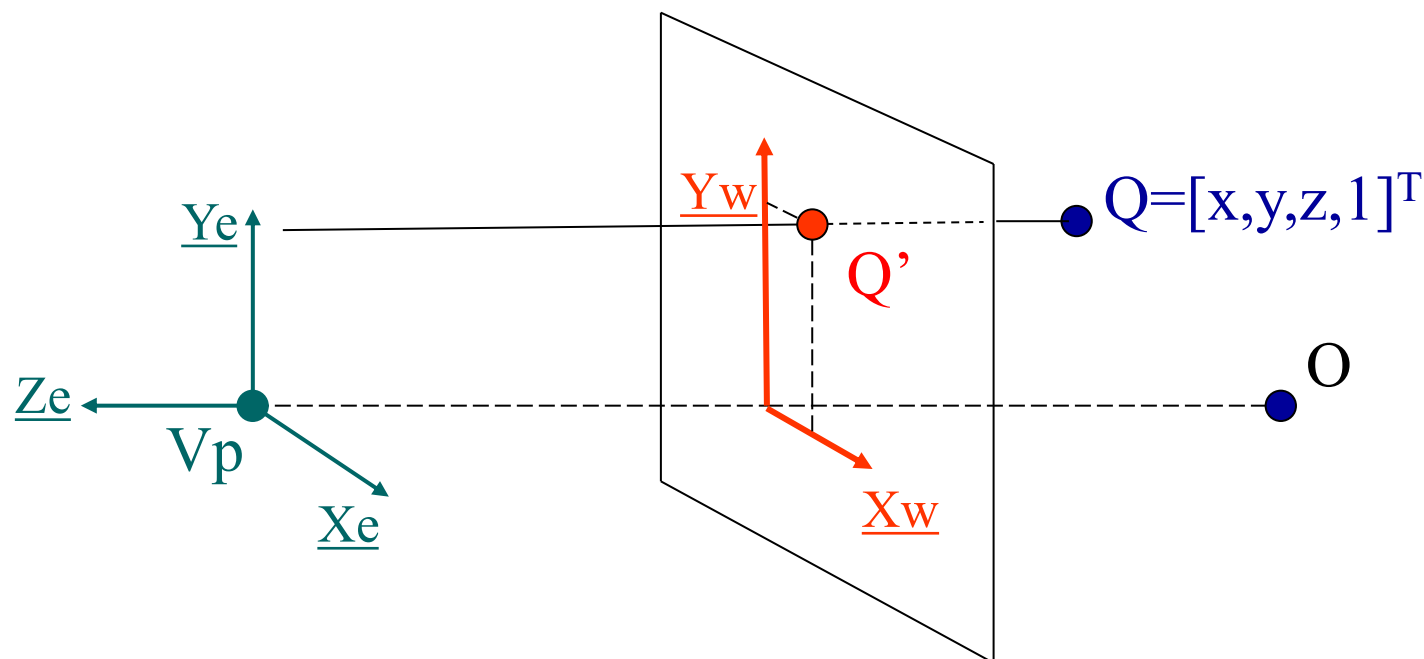
Esempio: Proiezione Ortogonale



Proiezione ortogonale, Orizzontale, Verticale, Laterale



Proiezione Geometrica Ortogonale

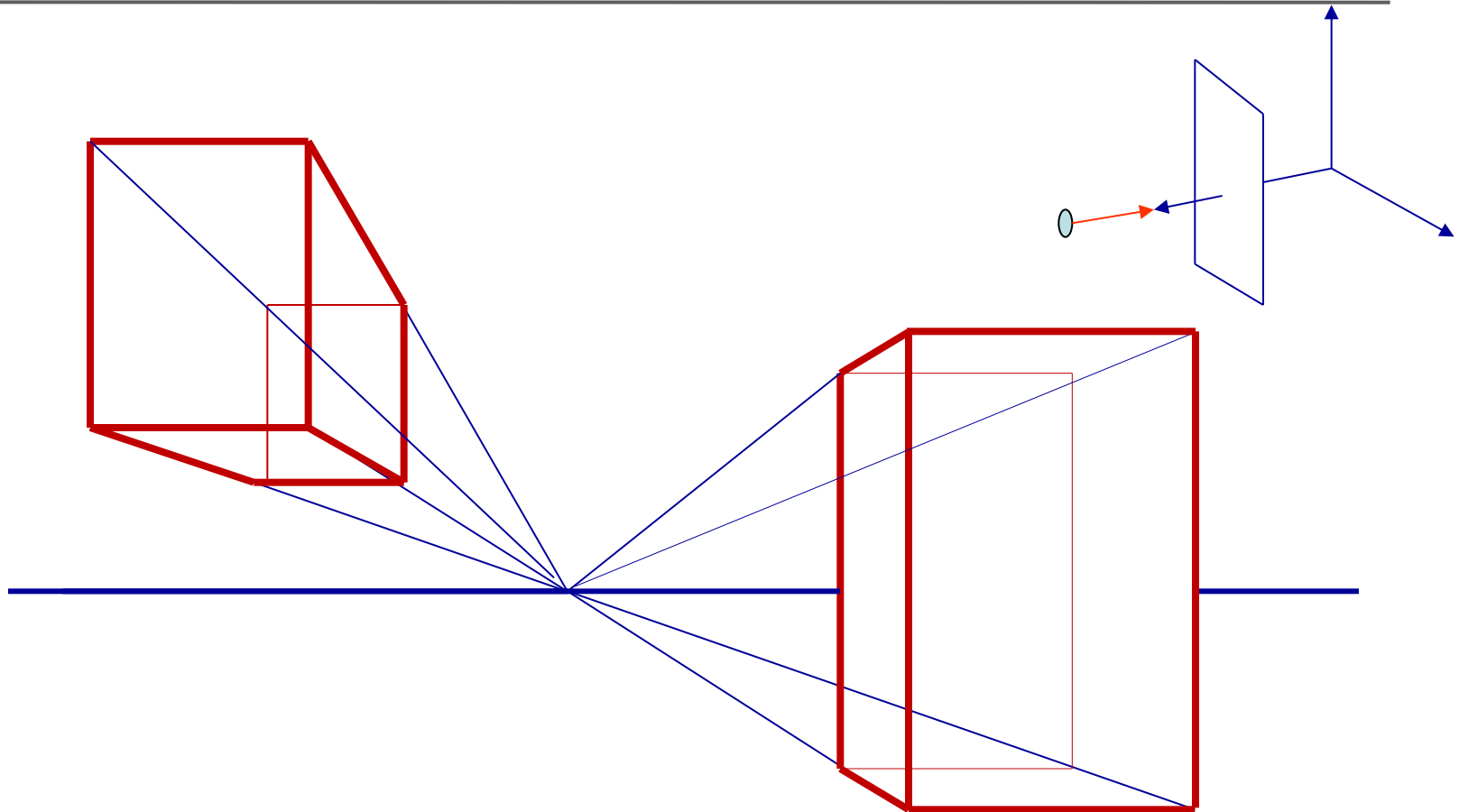


$$[x_e, y_e, z_e, 1]^T = V \cdot Q$$

$$x_w = x_e$$

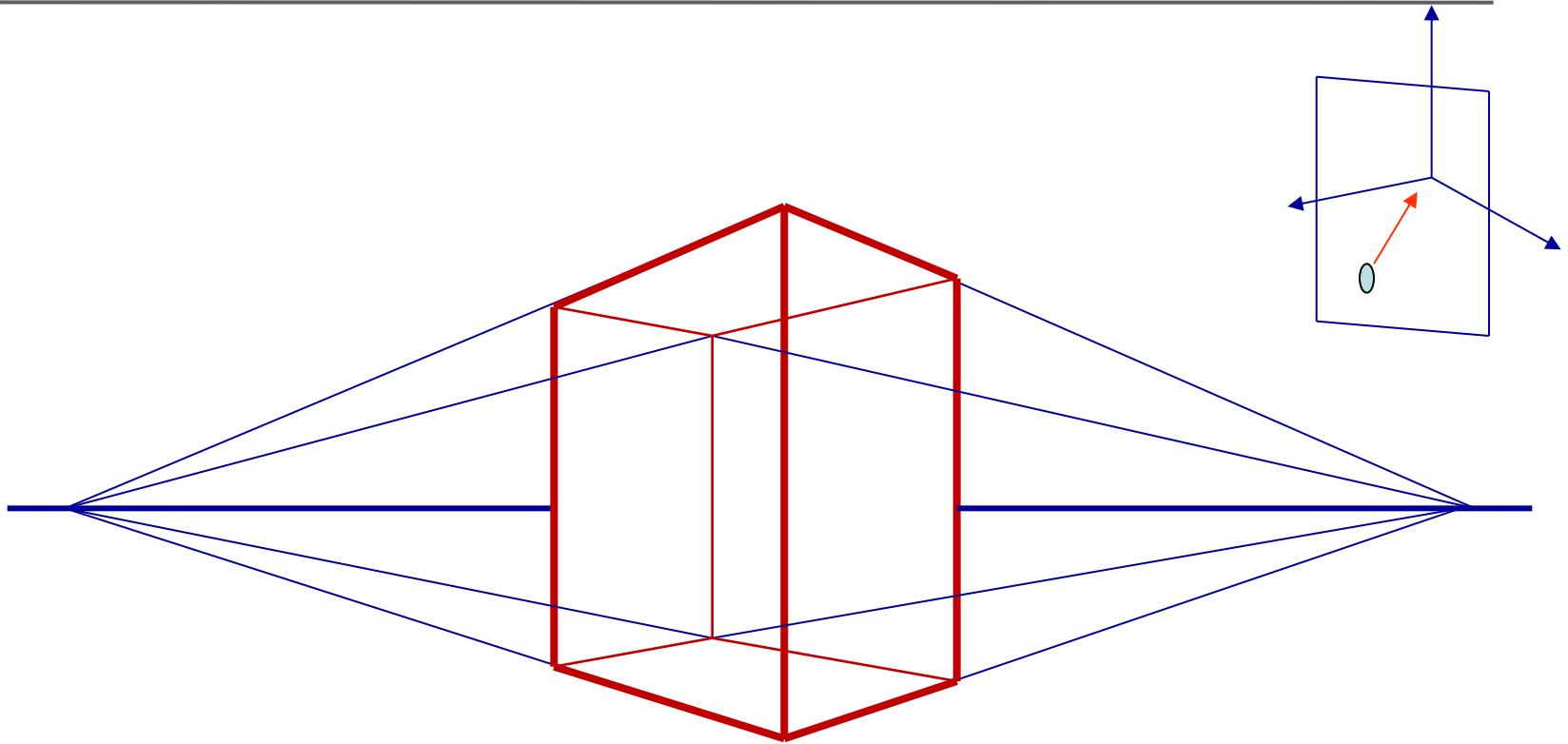
$$y_w = y_e$$

Proiezione prospettica a 1 Punto di Fuga



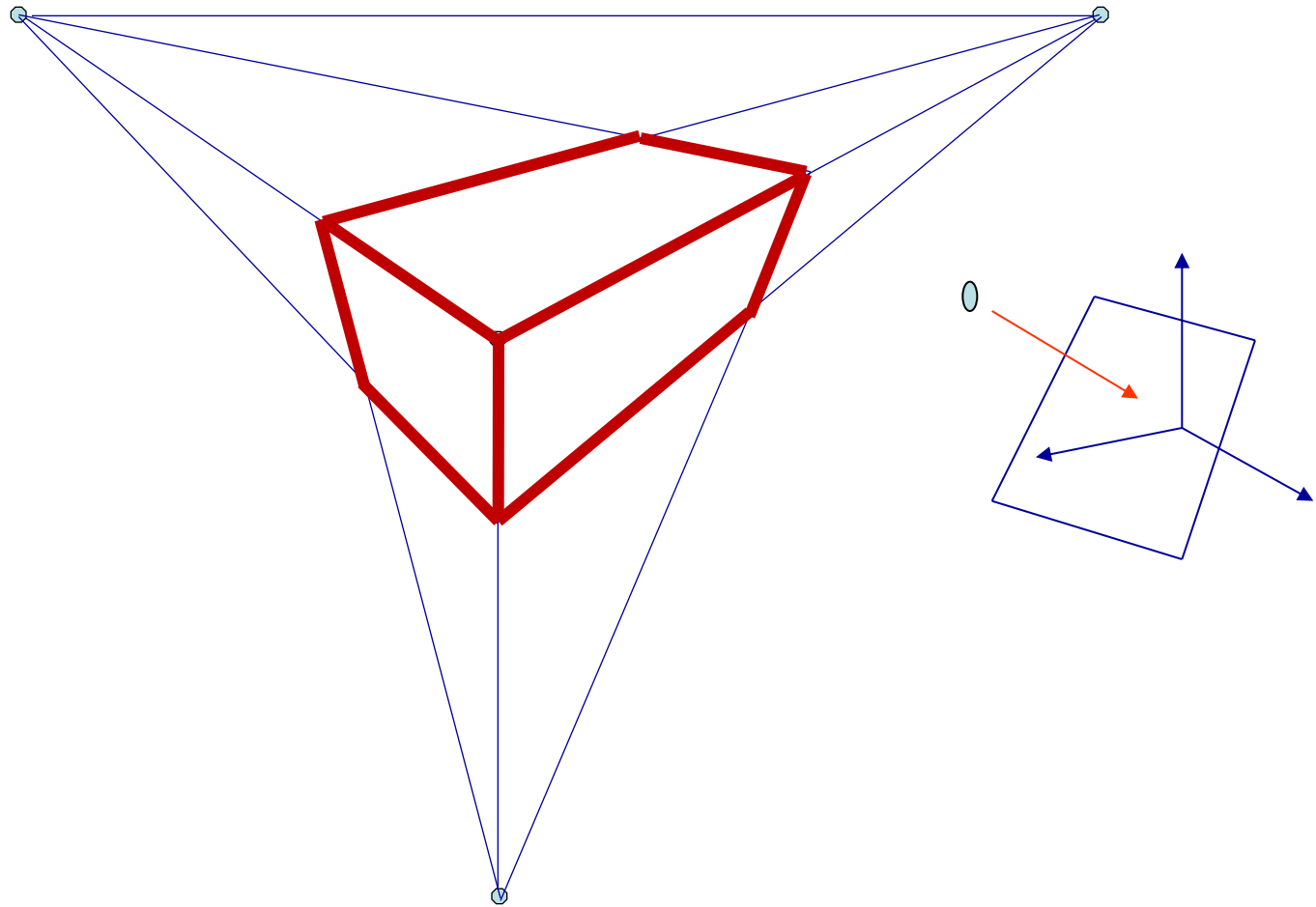
Piano di proiezione ortogonale ad uno degli assi e
quindi parallelo agli altri due

Proiezione prospettica a 2 Punti di Fuga



Piano di proiezione parallelo ad uno degli assi
coordinati

Proiezione prospettica a 3 Punti di Fuga



Piano di proiezione in posizione generica

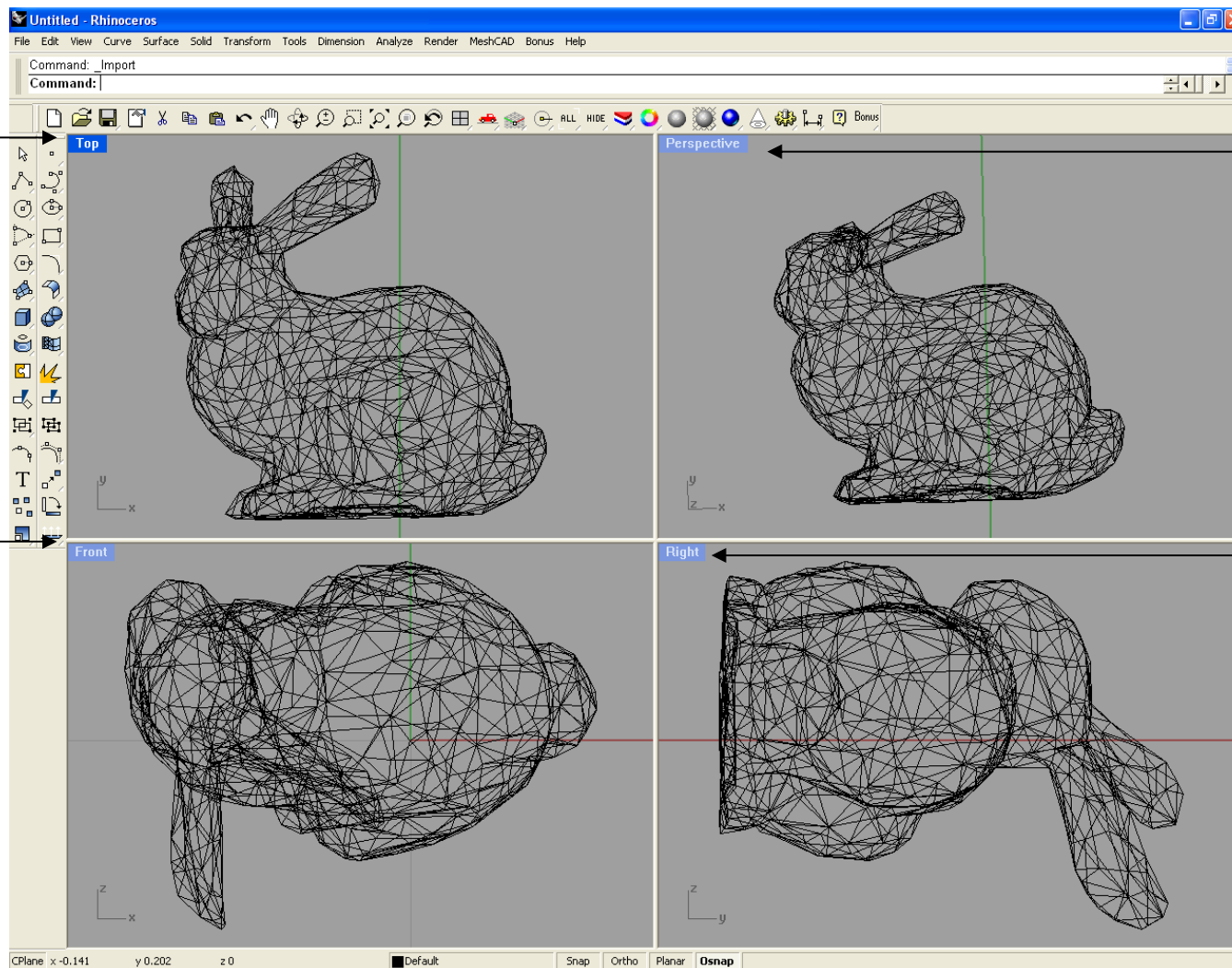
Esempio

Top
1 punto
di fuga

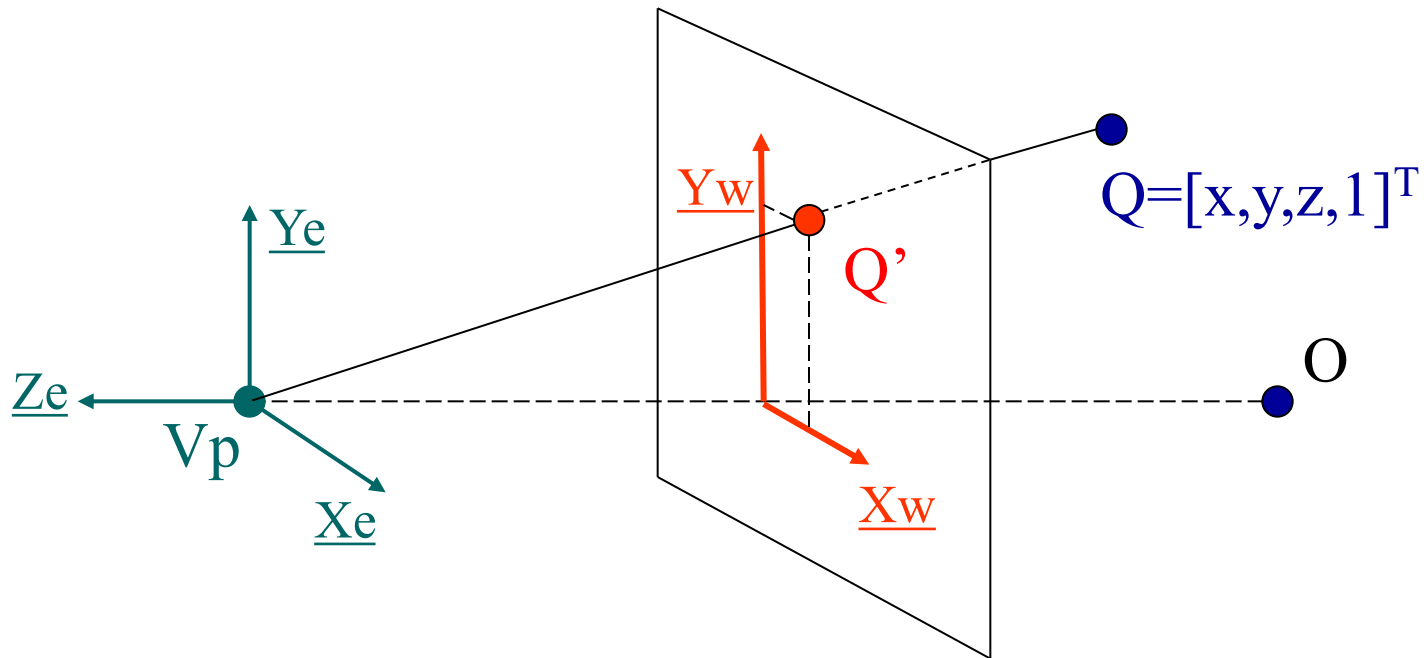
Front
1 punto
di fuga

Perspective
3 punti
di fuga

Right
1 punto
di fuga

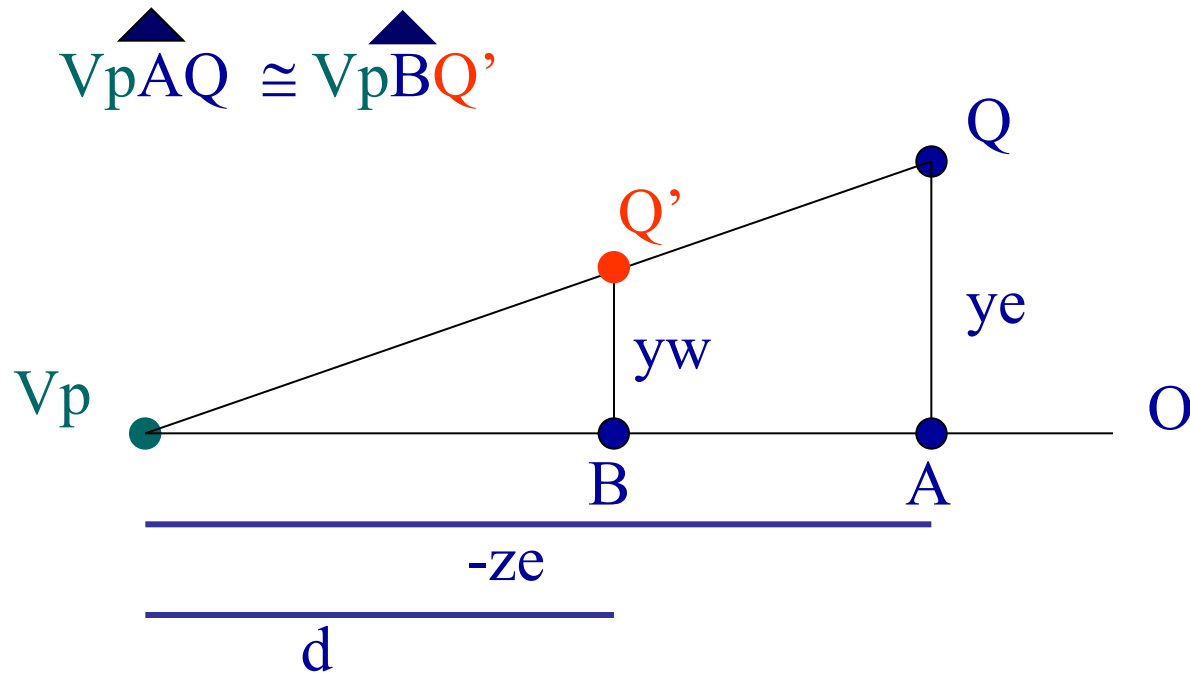


Proiezione prospettica a 3 punti di fuga



$$[x_e, y_e, z_e, 1]^T = V \cdot Q$$

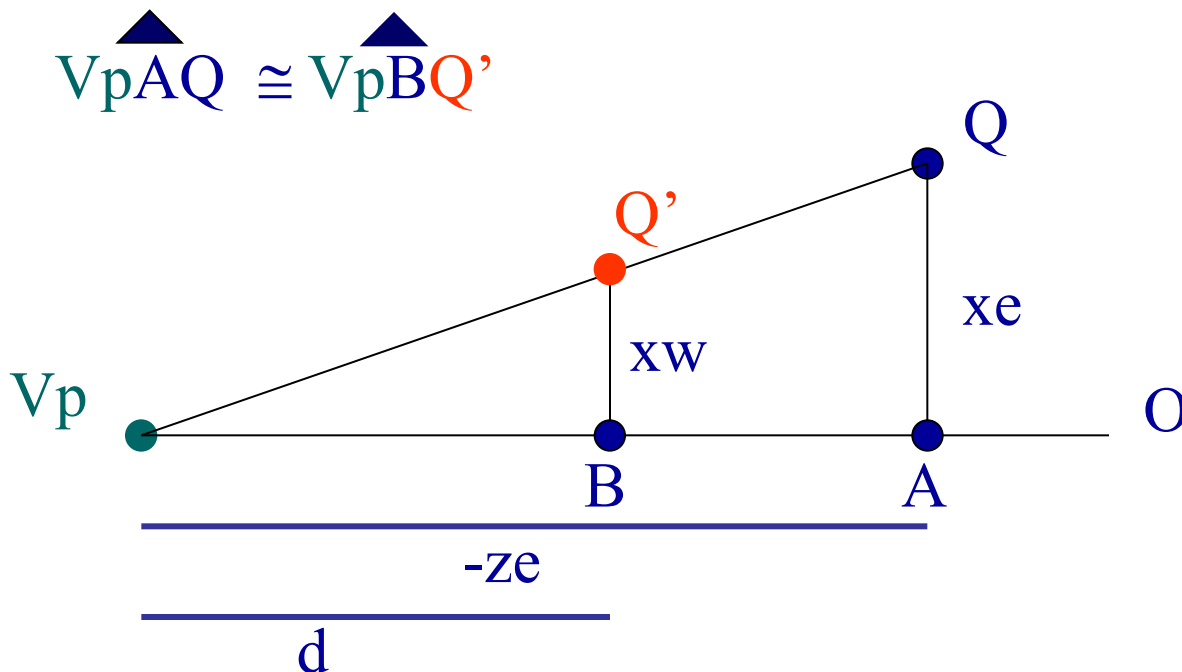
Proiezione prospettica a 3 punti di fuga



$$yw = \frac{d \cdot ye}{-ze}$$

Con d distanza fra
 V_p e il piano di
proiezione

Proiezione prospettica a 3 punti di fuga



$$xw = \frac{d \cdot xe}{-ze}$$

Con d distanza fra
 V_p e il piano di
proiezione



Proiezione prospettica a 3 punti di fuga o divisione prospettica

Indichiamo con DivPro l'applicazione che definisce la Proiezione Geometrica da 3D a 2D:

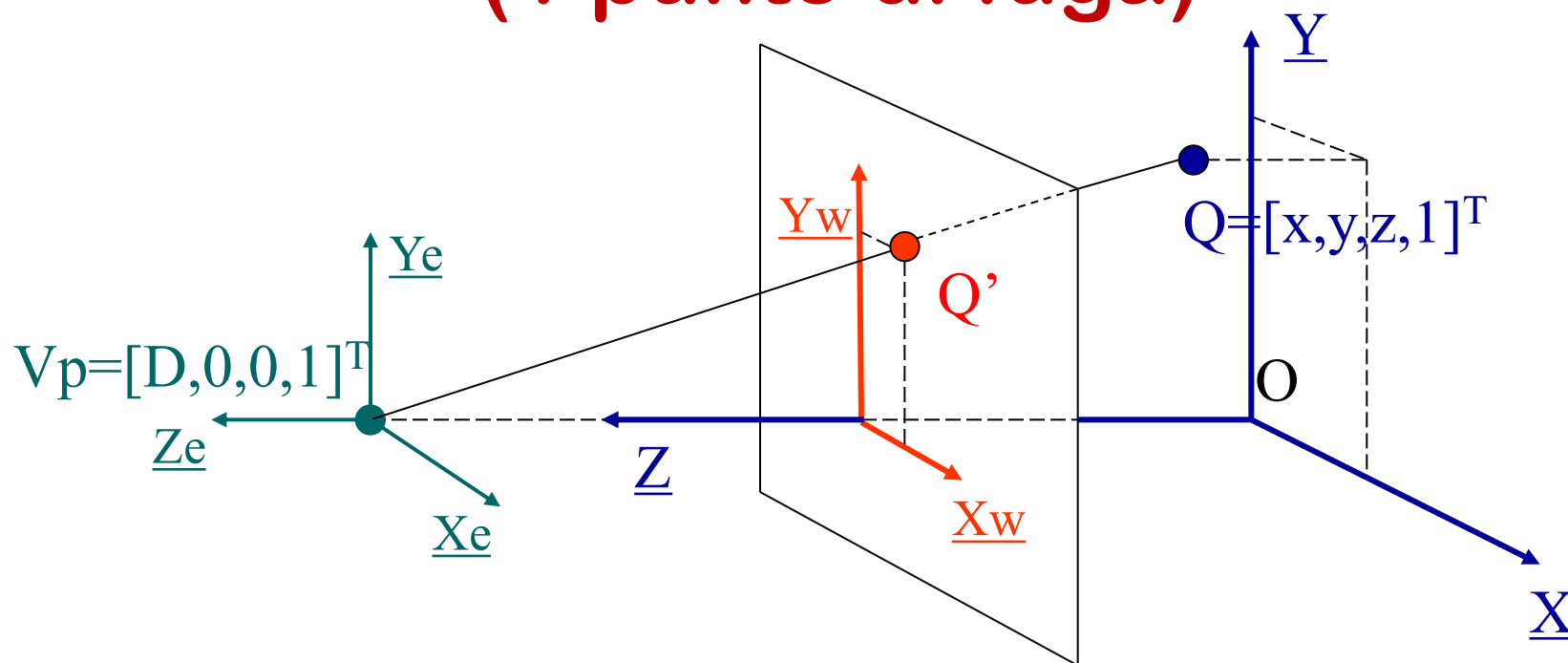
$$[x_w, y_w, 1]^T = \text{DivPro} ([x_e, y_e, z_e, 1]^T)$$

$$x_w = \frac{d \cdot x_e}{-z_e}$$

$$y_w = \frac{d \cdot y_e}{-z_e}$$

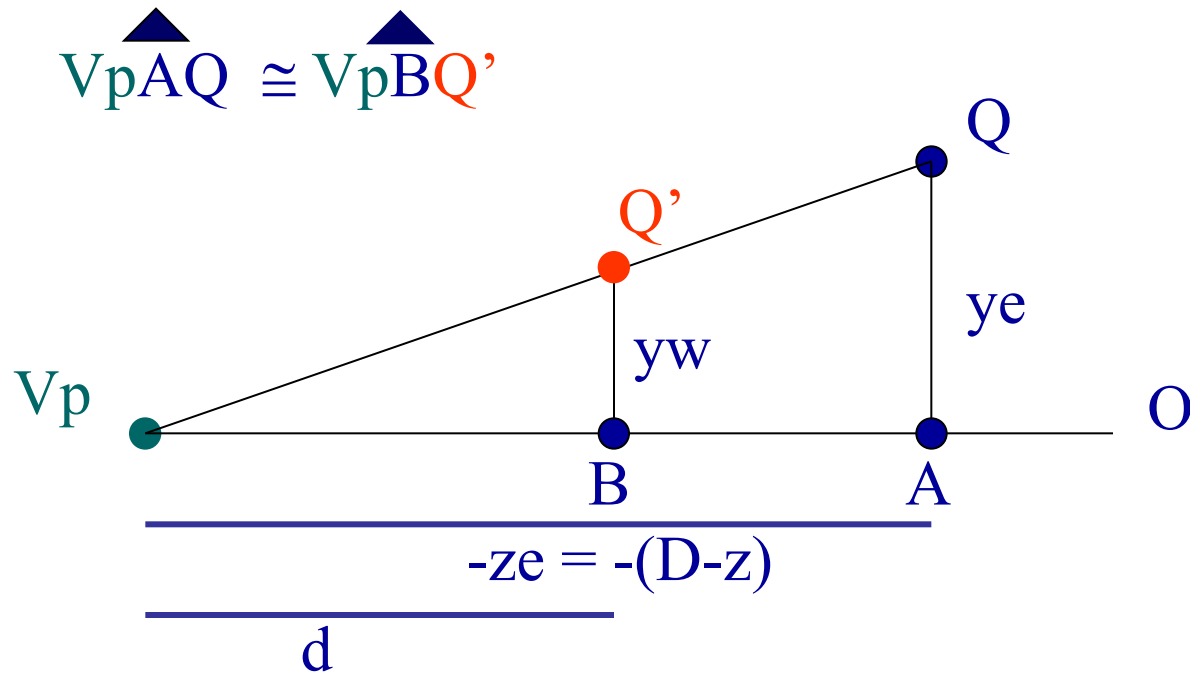


Trasformazione Sistema di Riferimento e Proiezione centrale (1 punto di fuga)



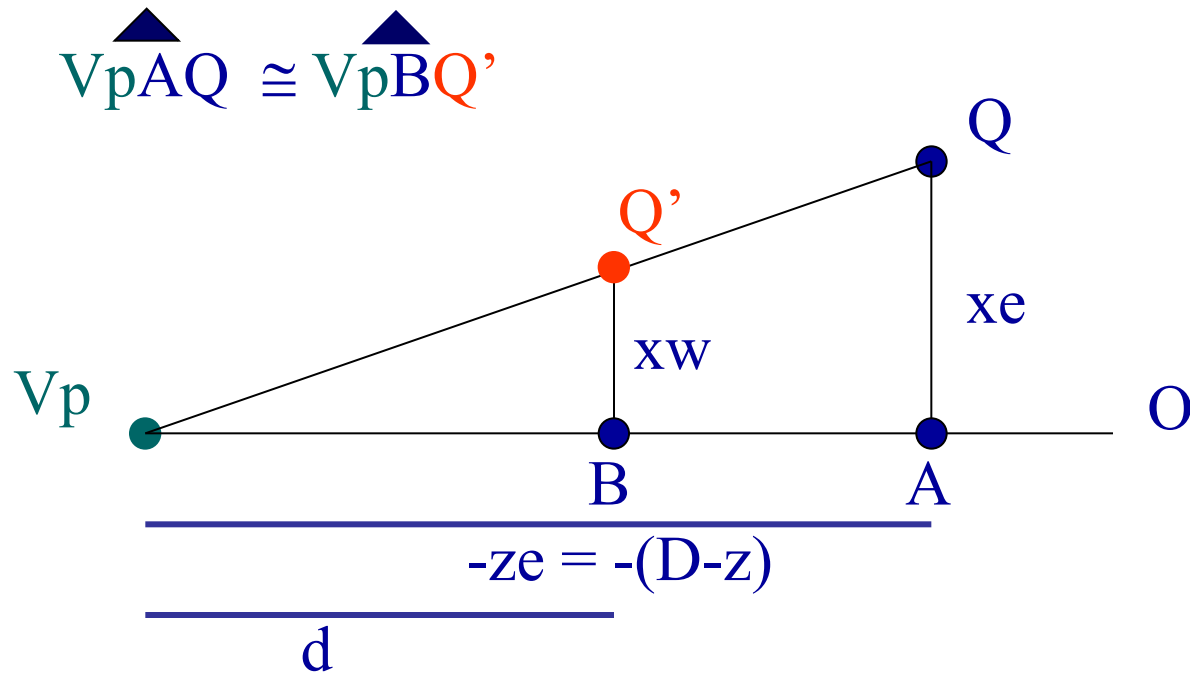
$$Q = [x_e, y_e, z_e, 1]^T = [x, y, -(D-z), 1]^T$$

Proiezione centrale (1 punto di fuga)



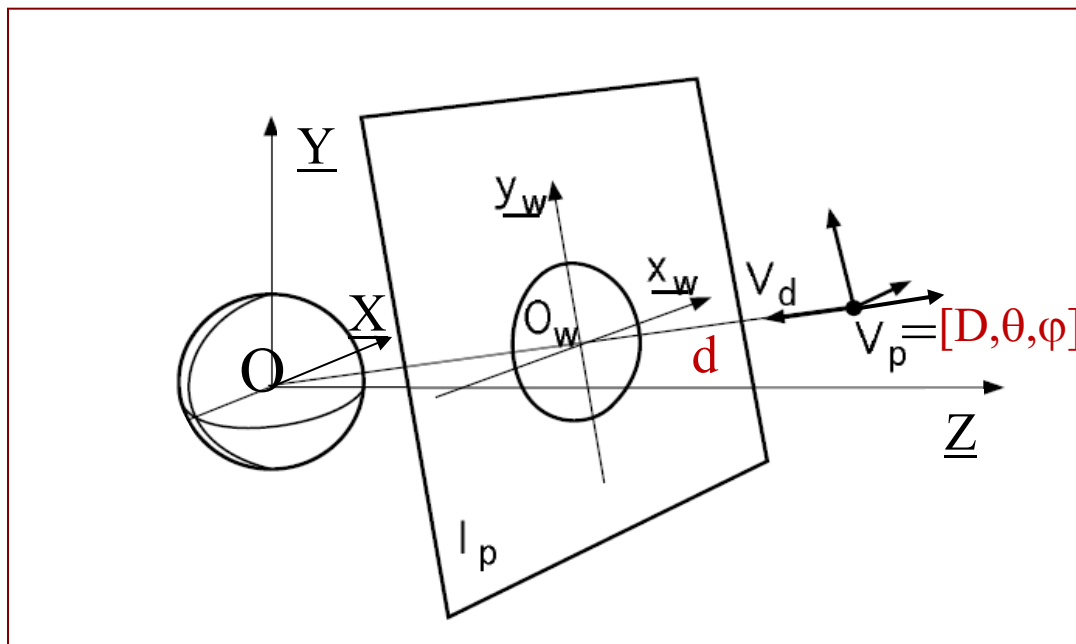
$$y_w = \frac{d \cdot y_e}{-ze} = \frac{d \cdot y}{-(D-z)}$$

Proiezione centrale (1 punto di fuga)



$$xw = \frac{d \cdot xe}{-ze} = \frac{d \cdot x}{-(D-z)}$$

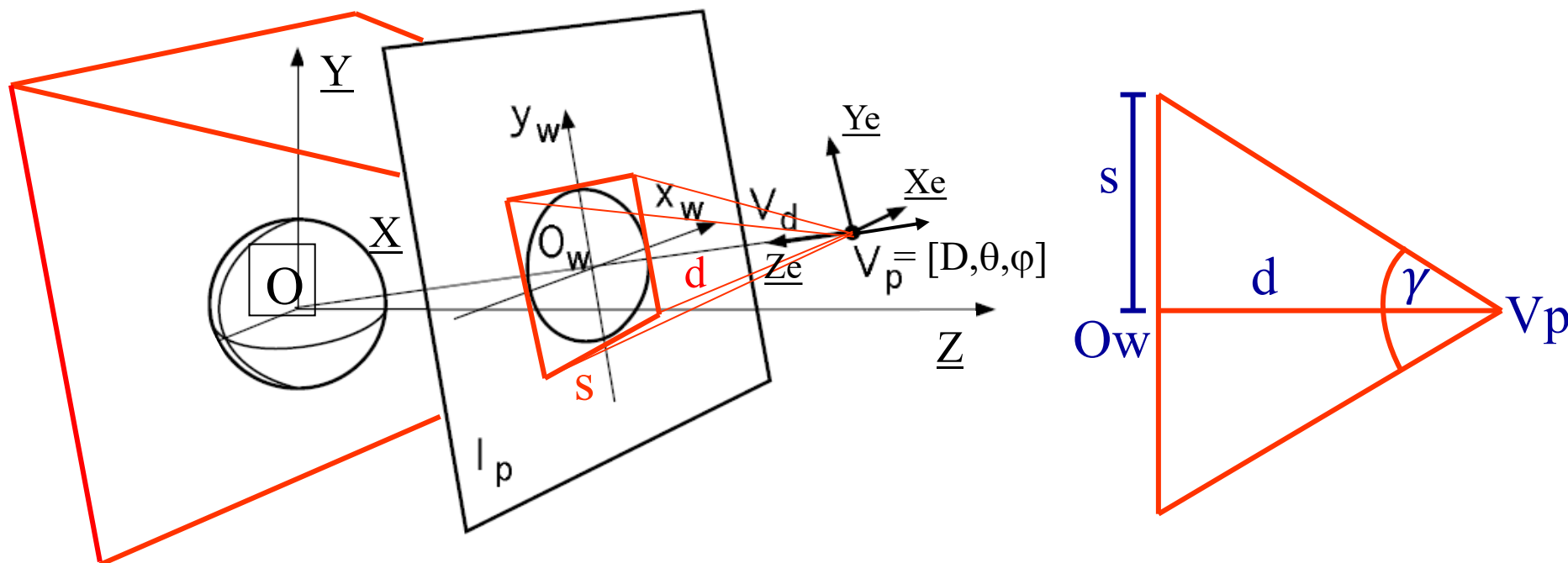
Parametri di Vista



- ❑ Variando θ e ϕ è possibile osservare un oggetto da differenti angolazioni;
- ❑ Variando D è possibile spostare l'osservatore vicino o lontano dall'oggetto;

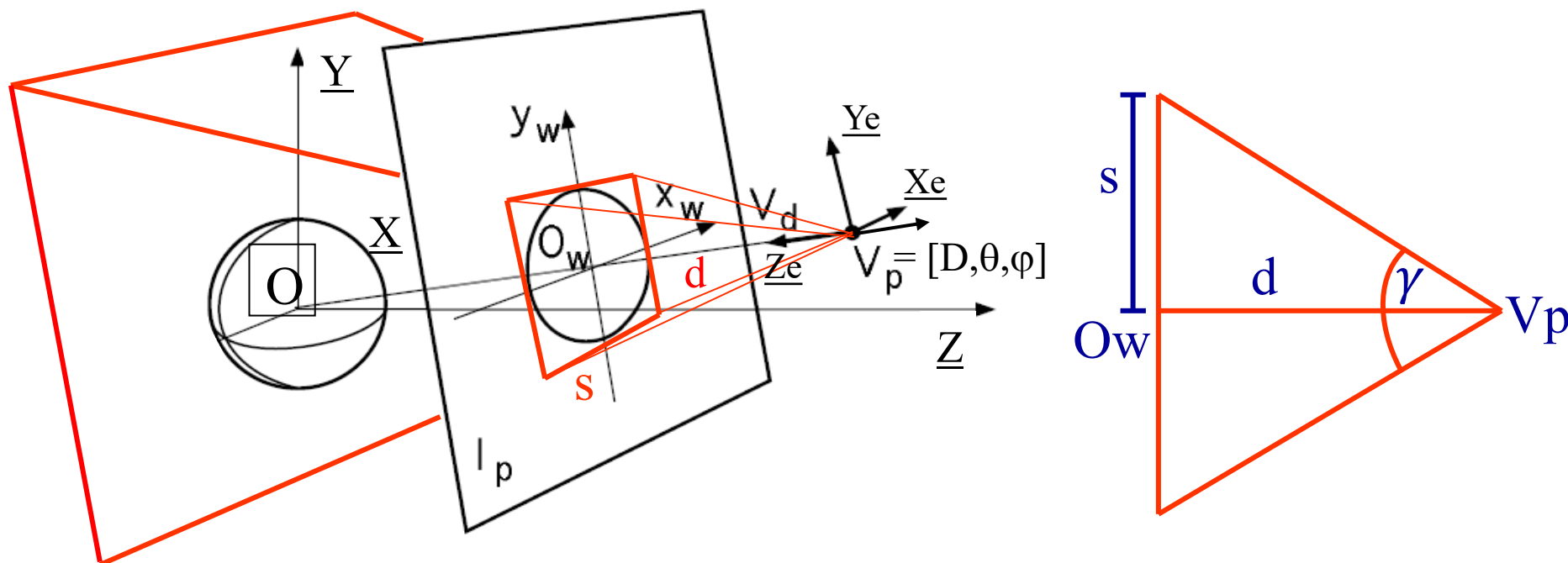
- ❑ Tenendo fissato V_p , variando d si modifica la dimensione dell'immagine proiettata, ma non della prospettiva;
- ❑ Tenendo fissato d , variando D si modifica la dimensione dell'immagine proiettata e la prospettiva,

Piramide di Vista e Window



l'Osservatore (V_p) guarda al centro dell'oggetto, ma con un'apertura angolare γ (cono di vista). Approssimiamo questo cono con una piramide che taglia il piano di proiezione definendo un rettangolo che rappresenta la window attraverso la quale si guarda il mondo.

Piramide di Vista e Window



Il lato s della window è definito dall'ampiezza della piramide di vista dell'osservatore in base alla semiampiezza angolare $\gamma/2$ e la distanza d dal piano di proiezione, da cui:

$$s = d \cdot \tan (\gamma/2)$$

Trasformazione di Scala

Senza perdere in generalità si applica la trasformazione di scala S a tutta la geometria in modo che la piramide di vista abbia un'apertura angolare $\gamma = 90$ gradi. Questo manterrà la window quadrata ma di dimensione $[-d, d] \times [-d, d]$.

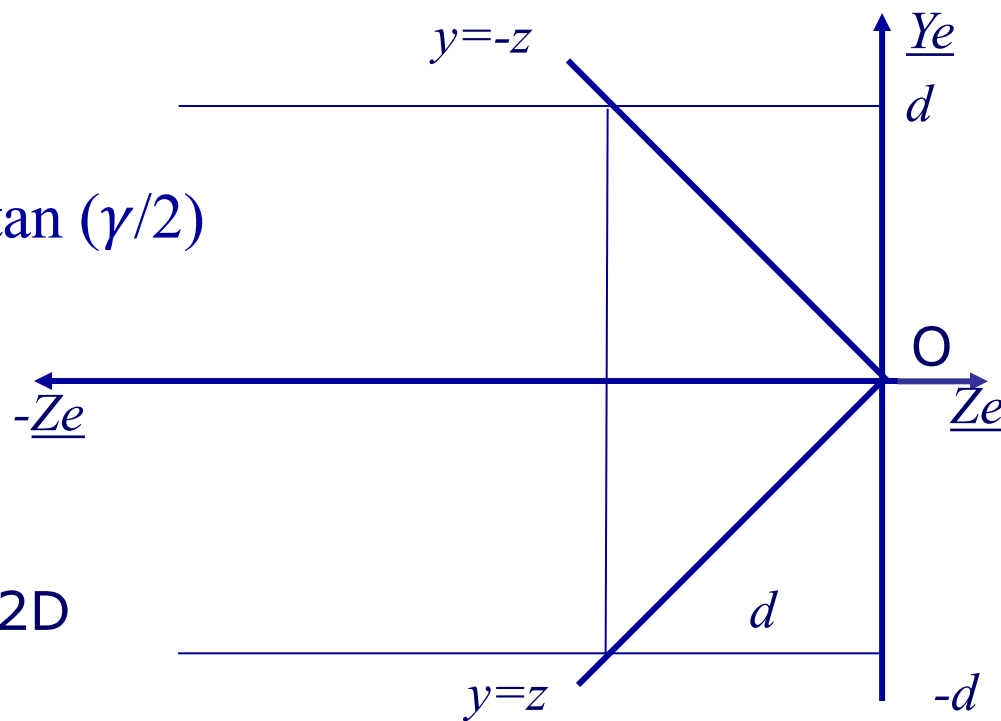
$$[x_e', y_e', z_e', 1]^T = S [x_e, y_e, z_e, 1]^T \quad \text{con} \quad S =$$

$$\begin{pmatrix} d/s & 0 & 0 & 0 \\ 0 & d/s & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

ed

$$s = d \cdot \tan(\gamma/2)$$

Vista 2D





Trasformazione di Scala

In realtà la matrice 4x4 da applicare ai punti, con le dette, ma che in più permette di deformare nella dimensione x la proiezione 2D in base all'aspect ratio della viewport è:

$$P = \begin{pmatrix} f/ar & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

P è detta
ProjectionMatrix

con

$f = d / s = 1 / \tan (\gamma/2)$ affinché l'apertura della piramide sia di 90 gradi

$ar = \text{aspect-ratio (Viewport)}$
 $= \text{width} / \text{height}$

affinché la proiezione 2D sia deformata in
base all'aspect-ratio della Viewport

Prospettiva

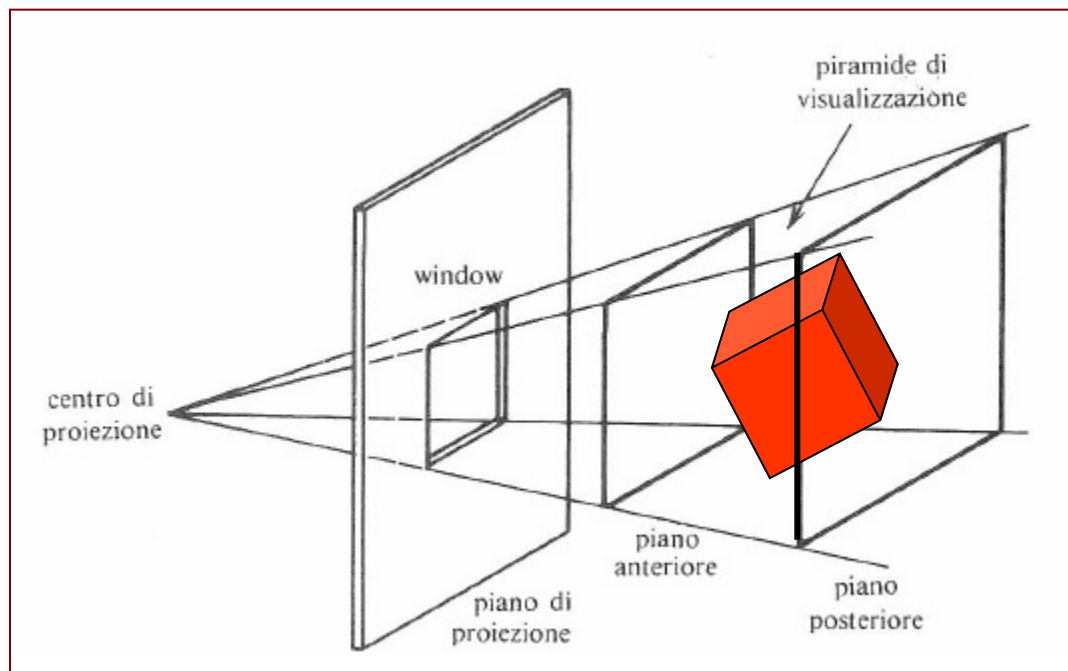
La matrice P è una matrice 4×4 che applicata a un punto produce un nuovo punto in coordinate omogenee, ma con la quarta componente diversa da 1 (matrice di Proiezione)

$$\begin{bmatrix} x_s \\ y_s \\ z_s \\ w_s \end{bmatrix} = P \cdot V \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = P \cdot \begin{bmatrix} x_e \\ y_e \\ z_e \\ 1 \end{bmatrix} = \begin{bmatrix} x_e \cdot d/s/ar \\ y_e \cdot d/s \\ 0 \\ -z_e \end{bmatrix}$$

La divisione per la quarta componente dà luogo alla proiezione prospettica, ma questa viene effettuata solo dopo il clipping.

Infatti in queste coordinate omogenee, dove l'ultima coordinata è diversa da 1, risulta più semplice applicare il clipping 3D rispetto alla **piramide di vista**.

Tronco di Piramide/Frustum



La determinazione di una piramide di vista con l'aggiunta di un **front plane** ed un **back plane** dà luogo ad un **tronco di piramide** detto **frustum**; solo le parti dell'oggetto interne a tale frustum saranno visibili e quindi successivamente da processare (**clipping 3D** di punti, linee e triangoli).



Libreria m4.js

Per definire la matrice di proiezione P useremo una function, della libreria **m4.js**, chiamata **perspective** con i seguenti parametri:

```
function perspective( fov, aspect, near, far ) {...}
```

fov : field of view, o campo di vista in radianti

aspect : aspect ratio della viewport (width / height)

near : near, o piano Z vicino

far : far, o piano Z lontano

Esempio:

```
// Compute the projection matrix  
var projectionMatrix m4.perspective(fieldOfViewRadians,  
                                     aspect, zNear, zFar);
```



Divisione Prospettica

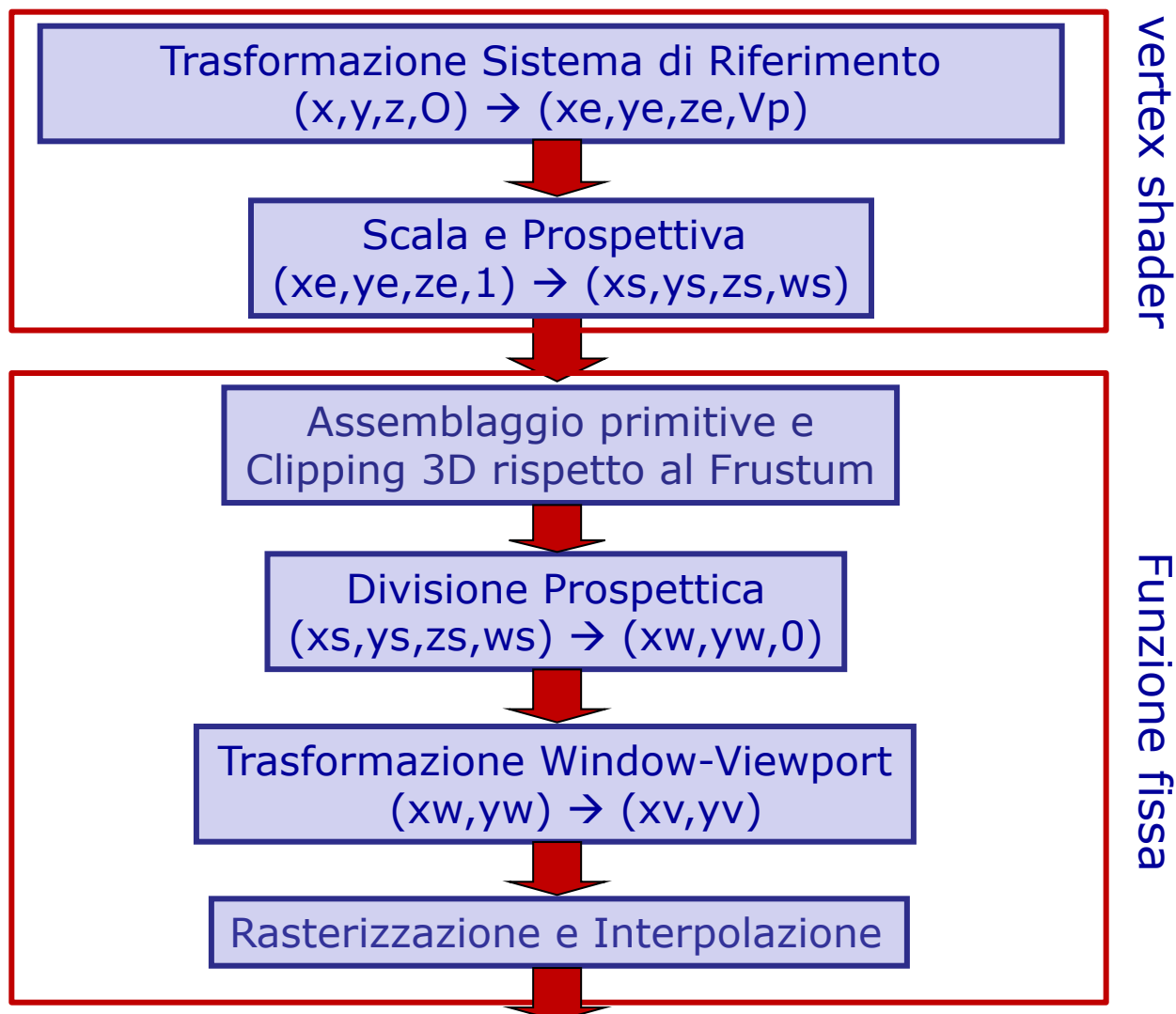
Come detto, solo dopo il clipping, si procede con la **divisione prospettica** che consiste nel dividere tutte le coordinate dei punti per la quarta coordinata, processo che corrisponde alla proiezione prospettica e che la window 2D sia $[-1,1] \times [-1,1]$:

$$\begin{bmatrix} xw \\ yw \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} xs / ws \\ ys / ws \\ 0 / ws \\ ws / ws \end{bmatrix} = \begin{bmatrix} -xe*d/s/ar / ze \\ -ye*d/s / ze \\ 0 \\ 1 \end{bmatrix}$$

Infine, si procede con la **trasformazione Window-Viewport** che porta i punti in coordinate intere schermo.



Pipeline di Rendering o di Vista





Codice di Esempio

Nella cartella `HTML5_webgl_2` vediamo insieme un codice che usa HTML5 + canvas + contesto webgl e implementa la trasformazione di vista analizzata:

vedi codice: `persp_cube_wire_webgl.html`

Questo codice utilizza le function `lookAt()` e `perspective()` della libreria `m4.js`.

Prima di procedere con l'Esempio vediamo qualche altro dettaglio sulla libreria `m4.js`



Function della Libreria m4.js

copy,
lookAt,
addVectors,
subtractVectors,
distance,
distanceSq,
normalize,
compose,
cross,
decompose,
mvec4,
dot,
identity,
transpose,
length,
orthographic,
frustum,

perspective,
translation,
translate,
xRotation,
yRotation,
zRotation,
xRotate,
yRotate,
zRotate,
axisRotation,
axisRotate,
scaling,
scale,
flatten,
multiply,
inverse,
transformVector,

transformPoint,
transformDirection,
transformNormal,
setDefaultType



m4.js e Typed Array

Le funzioni della libreria **m4.js** restituiscono per default array tipizzati e per la precisione **Float32Array**.

Gli **array tipizzati** sono stati progettati dal comitato degli standard WebGL, per motivi di prestazioni.

Gli **array Javascript** infatti sono generici e possono contenere oggetti, altri array ecc., e gli elementi non sono necessariamente sequenziali in memoria, come invece sono nel linguaggio C.

WebGL richiede che i buffer siano sequenziali in memoria, perché è così che GLSL, basato sul linguaggio C, si aspetta che siano.



m4.js e Typed Array

Nota: alcune funzioni come `array.push` non funzionano bene su array tipizzati. E' per questo che si usa la libreria `m4.js` che comunque lavora e restituisce, su richiesta, anche array non tipizzati.

```
var t1 = m4.subtractVectors(vertices[b], vertices[a]);
var t2 = m4.subtractVectors(vertices[c], vertices[b]);
//restituisce un typed array Float32Array
normal = m4.cross(t1, t2); //restituisce un array
                           //tipizzato
```

```
var t1 = m4.subtractVectors(vertices[b], vertices[a]);
var t2 = m4.subtractVectors(vertices[c], vertices[b]);
//restituisce un JavaScript array
var normal=[ ];
normal = m4.cross(t1, t2, normal); //restituisce un array
                                   //non tipizzato
```

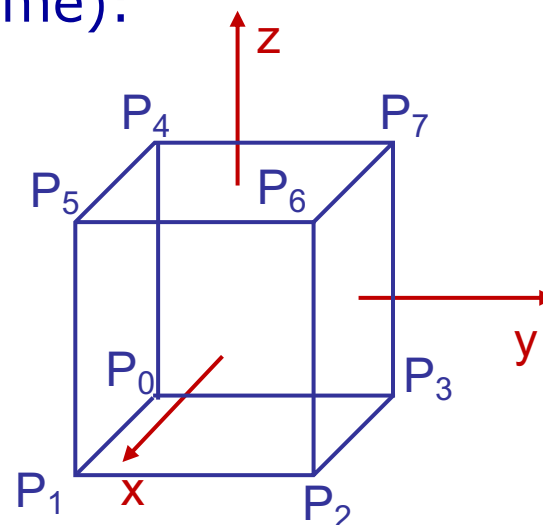
Esempio:

`persp_cube_wire_webgl.html`

Cominciamo con la definizione della geometria.
Viene definito un cubo mediante i vertici (in coordinate omogenee) e i lati della facce quadrate (si vuole disegnare un cubo wire-frame):

```
vertices=[  
-1,-1,-1,1, 1,-1,-1,1, 1,1,-1,1,  
-1,1,-1,1, -1,-1,1,1, 1,-1,1,1,  
1,1,1,1, -1,1,1,1];
```

```
indices = [  
0,1, 1,2, 2,3, 3,0, 4,5, 5,6,  
6,7, 7,4, 0,4, 1,5, 2,6, 3,7];
```



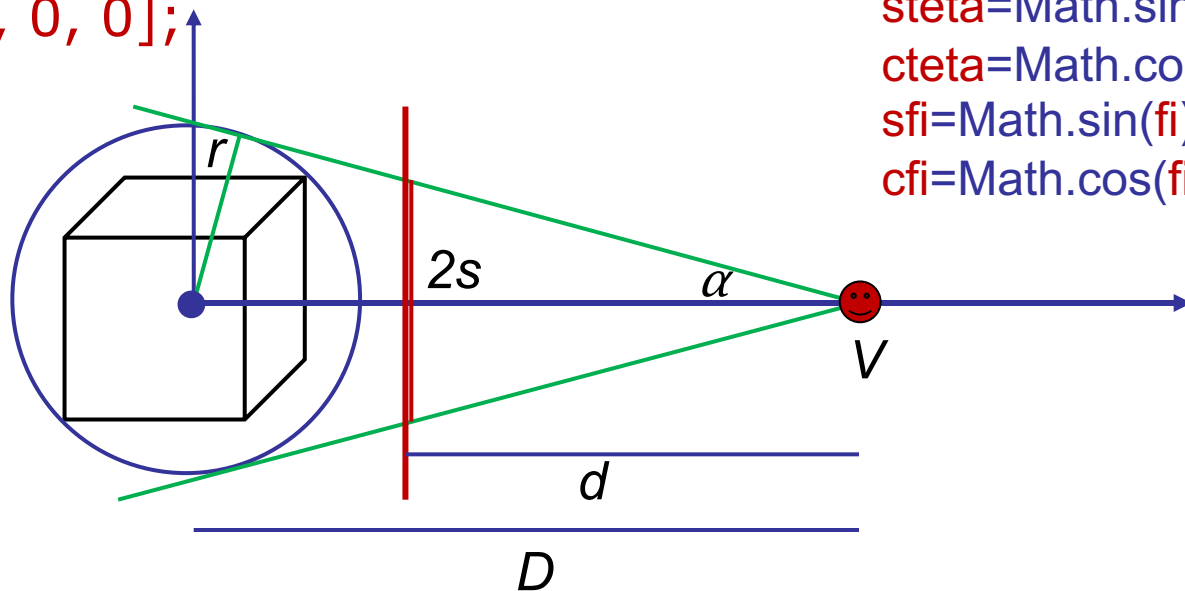
Definizione Camera

Vengono definiti i parametri di vista (posizione camera, View-up, centro di vista) in modo tale da garantire che il cubo venga interamente visualizzato all'interno del canvas.

```
teta=degToRad(20), fi=degToRad(60), D=5;
Camera = [D*sfi*cteta, D*sfi*steta, D*cfi];
View-up = [0, 0, 1];
Target = [0, 0, 0];
```

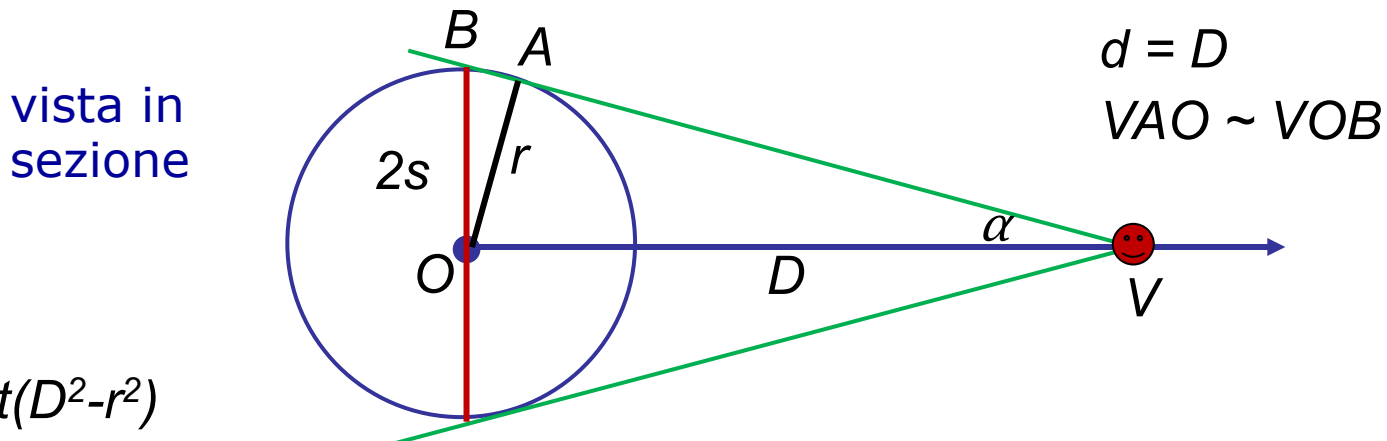
con:

```
steta=Math.sin(teta);
cteta=Math.cos(teta);
sfi=Math.sin(fi);
cfi=Math.cos(fi);
```



Definizione Camera

Per semplicità e senza perdere in generalità si assume che $d=D$ (cioè si prende il piano di proiezione passante per l'origine del sistema dell'oggetto), inoltre vogliamo determinare α in modo da vedere, inizialmente, l'intero cubo.



$$AV = \sqrt{D^2 - r^2}$$

$$OB : OA = OV : AV$$

$$OB = s = r * D / \sqrt{D^2 - r^2}$$

$$\alpha = 2 \operatorname{atan} (s / D)$$

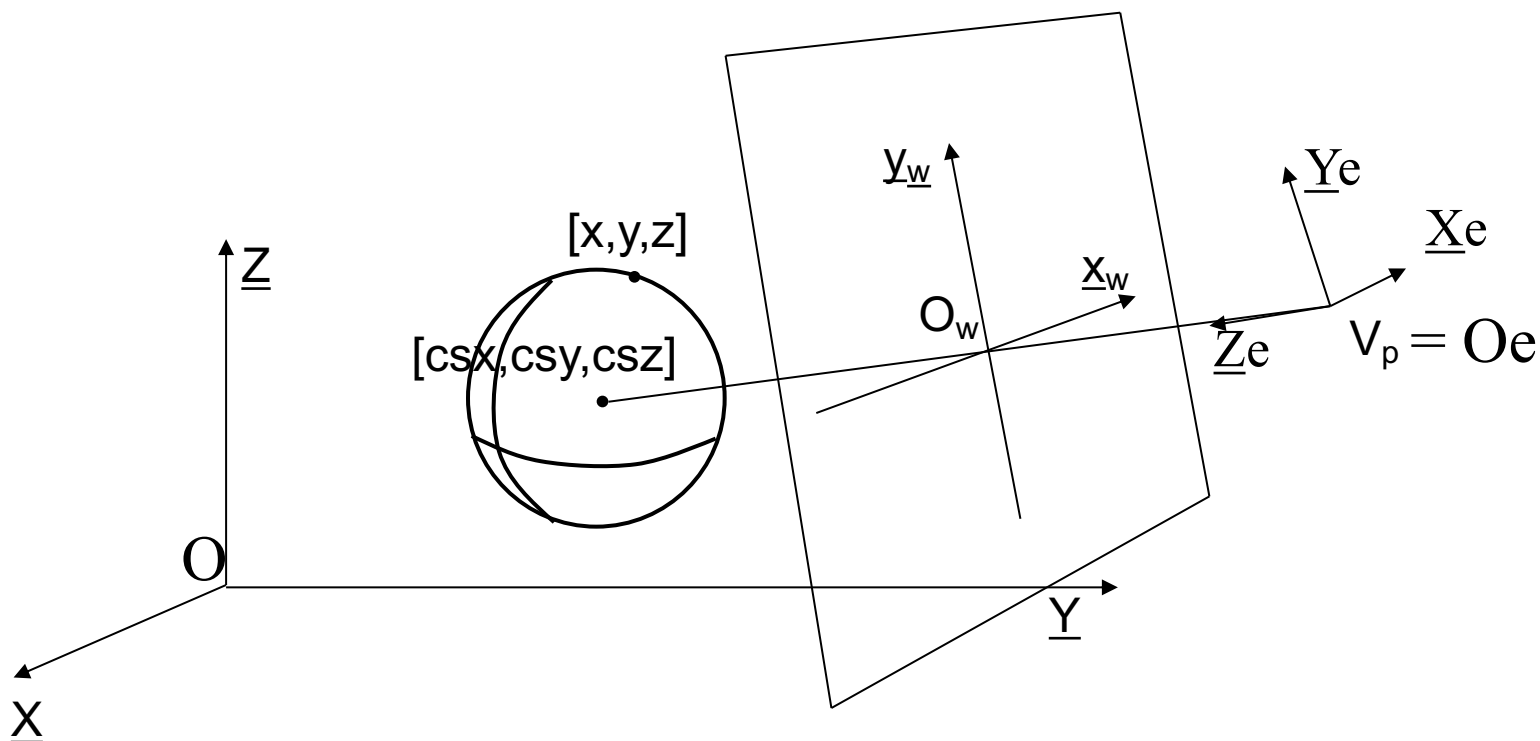


function render()

Questa function viene chiamata all'inizio, e poi richiamata ed eseguita ogni volta che viene cambiato un parametro di vista; nell'ordine calcola:

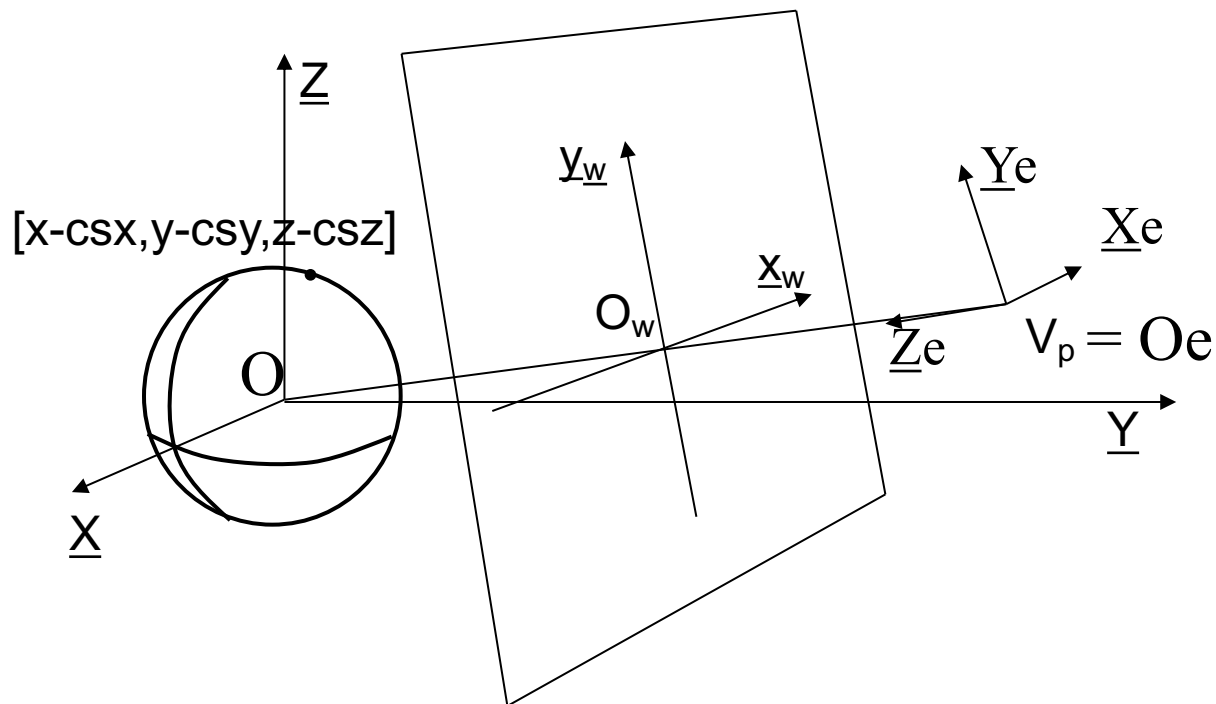
- Eventuale Matrice M di Trasformazione oggetto nell'origine
- Matrice P di trasformazione prospettica
- Matrice V di trasformazione di Vista
- Matrice prodotto $PVM = P * V * M$
- Applicazione di PVM ai vertici
- Caricamento dei vertici trasformati nel vertex_buffer su GPU
- Definisce la viewport
- Inizializza il frame buffer
- Lancia la pipeline

Trasla l'oggetto nell'origine



L'Osservatore guarda l'oggetto (il centro dell'oggetto),
ma se l'oggetto non è centrato nell'Origine? allora ...

Trasla l'oggetto nell'origine



... allora si trasli l'oggetto (i vertici dell'oggetto) nell'origine del sistema di coordinate del mondo; così facendo ci si riconduce alla situazione vista precedentemente.



Esercizi

Modificare il codice `persp_cube_wire_webgl.html` per:

1. sperimentare differenti View-Up vector;
2. gestire anche con il mouse i parametri di vista (angoli θ e φ e distanza D rispettivamente usando move e rotella) (sol. codice `persp_cube_wire_interact_webgl.html`);
3. sperimentare il clipping 3D rispetto al frustum;
4. sperimentare oltre la proiezione prospettica quella ortografica ed il frustum relativo
5. gestire più oggetti (cubo) mediante trasformazioni geometriche



Proposte di Esercizi

Sulla base dei codici nella cartella HTML5_webgl_2 realizzare il seguente percorso (vedi anche il file Readme):

- cubo a linee con parametri di vista(bottoni)
([persp_cube_wire_webgl.html](#))
- come il precedente, ma in contesto 2d
([persp_cube_wire_2d.html](#))
- cubo a linee con parametri di vista (bottoni e mouse:
[persp_cube_wire_interact_webgl.html](#))
- come il precedente, ma in contesto 2d
([persp_cube_wire_interact_2d.html](#))
- cubo a linee con parametri di vista (bottoni e mouse) e
webglutils ([persp_cube_wire_interact_webglutils.html](#))
(vedi info su libreria wenglutils)
- rotazione cubo a linee ([cube_wire_interact_webgl.html](#))
- come il precedente, ma in contesto 2d
([cube_wire_interact_2d.html](#))



Libreria webgl-utils.js

La libreria `webgl-utils.js` mette a disposizione alcune funzioni utili per compilare, linkare e ottenere uno shader program.

```
function createProgramFromScripts(gl, shaderScriptIds, ... ) {  
...  
}
```

`shaderScriptIds` :Array of ids of the script tags for the shaders. The first is assumed to be the vertex shader, the second the fragment shader.

```
// setup GLSL program  
var program = webglUtils.createProgramFromScripts(gl,  
    ["3d-vertex-shader", "3d-fragment-shader"]);
```



Libreria webgl-utils.js

La libreria **webgl-utils.js** propone anche altre funzioni utili:

createAugmentedTypedArray,
createAttribsFromArrays,
createBuffersFromArrays,
createBufferInfoFromArrays,
createAttributeSetters,
createProgram,
createProgramFromScripts,
createProgramFromSources,
createProgramInfo,
createUniformSetters,
createVAOAndSetAttributes,
createVAOFromBufferInfo,
drawBufferInfo,
drawObjectList,
glEnumToString,

getExtensionWithKnownPrefixes,
resizeCanvasToDisplaySize,
setAttributes,
setBuffersAndAttributes,
setUniforms

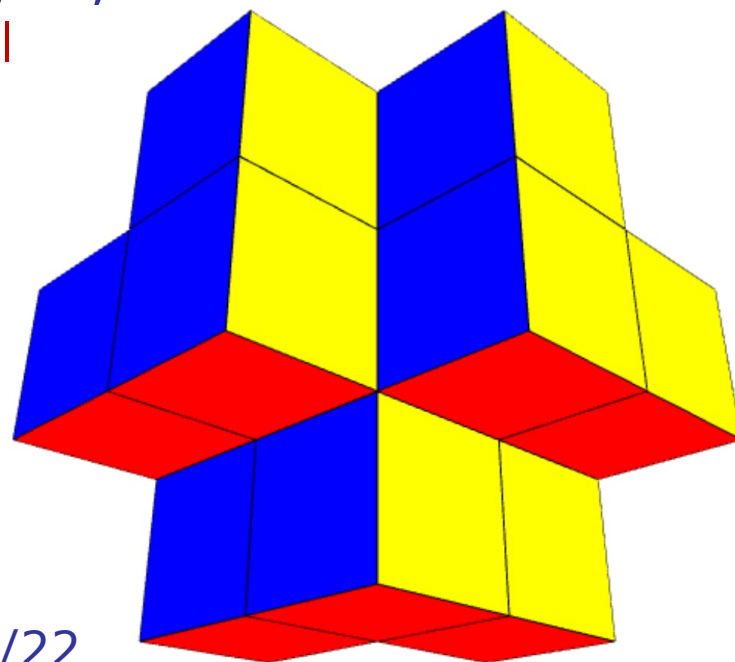
Esercizio 2

Modificare il codice

`persp_cube_wire_interact_webgl.html`

per progettare, visualizzare e interagire con un oggetto 3D composto da cubi colorati ed in particolare lo si utilizzi per realizzare il logo del corso A.A.2021/22; il codice si chiami: `persp_cubes_interact_webgl`

Nota: si osservi che le facce oltre ad essere colorate sono marcate dal disegno dei lati; questo serve nel caso si vogliano disegnare i cubetti, e quindi l'oggetto 3D, con un solo colore.



logo A.A.2021/22



Esercizio 3

Si veda ed esamini il codice

`persp_wire_interact_obj_webgl.html`

che carica un oggetto mesh in formato .obj Wavefront e lo disegna in wire-frame:

Modificare il codice in oggetto per realizzare l'animazione di un oggetto sfera che si muove e rimbalza contro le pareti di una stanza a forma di cubo (generalizzare il codice `ball3.html`; si mantengano i parametri di vista tramite il menù a bottoni e l'interazione con il mouse)

Formato OBJ

https://en.wikipedia.org/wiki/Wavefront_.obj_file

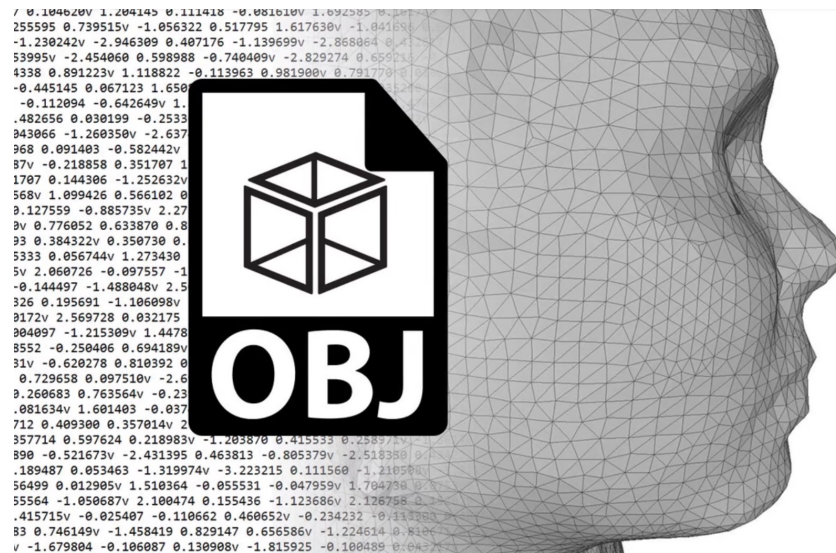
E' un formato molto diffuso della Alias-Wavefront (ora Autodesk). Puo' essere in binario o in ASCII (testo). Permette la memorizzazione di mesh 3D e varie info utili per il loro disegno:

```
# Blender v3.0 (sub 0) OBJ File: ''
# www.blender.org
```

```
# 8 vertices
```

```
v 1.000000 -1.000000 1.000000
v 1.000000 1.000000 1.000000
v -1.000000 1.000000 1.000000
v -1.000000 -1.000000 1.000000
v -1.000000 1.000000 -1.000000
v -1.000000 -1.000000 -1.000000
v 1.000000 1.000000 -1.000000
v 1.000000 -1.000000 -1.000000
```

```
...
```





Formato OBJ (continua)

```
...  
# 8 texture coordinates  
vt 0.5 0.5  
...  
# 8 normal vertices  
vn 1.0 0.0 0.0  
...  
  
# 6 faces  
  
f 1/1/4 2/2/1 3/3/2 4/4/3  
f 4/4/4 3/3/1 5/5/2 6/4/3  
f 7/7/4 5/5/1 3/5/2 2/2/3  
f 1/1/4 8/8/1 7/7/2 2/2/3  
f 6/6/4 8/8/1 1/1/2 4/4/3  
f 6/6/4 5/5/1 7/7/2 8/8/3
```

Indice coord. vertice

f v/vt/vn v/vt/vn v/vt/vn

Indice coord. texture

Indice coord. normale



WebGL 2.0:

Qualificatori Attribute e Varying

WebGL 2.0 si basa sulla versione OpenGL ES 3.0 (che usa lo shader preprocessor #version 300) e in questa i qualificatori **attribute** e **varying** sono stati sostituiti dai qualificatori **in** e **out**. Nel programma applicativo non cambia nulla.

Esempi di **Vertex** shader:

```
# version 100 es
attribute vec3 vPosition;
attribute vec3 vColor;
varying vec3 color;
void main(void) {
    gl_Position =vec4(vPosition, 1.0);
    color = vColor;
}
```

```
# version 300 es
in vec3 vPosition ;
in vec3 vColor;
out vec3 color;
void main(void) {
    gl_Position =vec4(vPosition, 1.0);
    color = vColor;
}
```



WebGL 2.0:

Qualificatori Attribute e Varying

ed esempi di **Fragment Shader** :

```
# version 100 es
varyng vec3 color;
void main(void) {
    gl_FragColor = vec4(color, 1.0);
}
```

```
# version 300 es
in vec3 color;
out vec4 FragColor;
void main() {
    FragColor = vec4(color, 1.0);
}
```



Esempio

Vediamo un codice WebGL2 che visualizza un cubo colorato:
si tratta dello stesso esempio:

`HTML5_webgl_2/cube_color_interact_webgl.html`

ma in WebGL2:

`HTML5_webgl_2/cube_color_interact_webgl2.html`



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Giulio Casciola

Dip. di Matematica

giulio.casciola@unibo.it